

Hybrid-INoT: a hybrid agent architecture with internal role segregation for engineering tasks under long shared context

Daniil Privezentsev^{1*}

^{1*}National Research University Higher School of Economics, Moscow, Russia.

Corresponding author(s). E-mail(s): daprivezentsev@edu.hse.ru;

Abstract

External multi-agent orchestration repeatedly transmits shared context to separate large-language-model roles, increasing token usage. We propose **Hybrid-INoT**, a hybrid system in which planner, worker, and critic roles share one model call, whereas verification and tool execution remain external. An artifact-backed evaluation compares the complete B3 package—combined call plus deterministic context compression above 4,096 tokens—with an uncompressed three-call B2 baseline. In a real Gemini 3.1 Pro Preview API pilot ($n = 5$ unique tasks) B3 reduced mean token consumption by 55.6–88.9% over target contexts of 256–16,384 tokens; an independent Gemini 3.1 Flash-Lite replication ($n = 20$, seed 42) yielded 49.6–90.3% savings. For Flash-Lite, mean within-task U_{tok} gains were 106.5, 144.8, 183.3, and 934.6%, and every observed pair exceeded 15%; a conditional signed-rank sensitivity analysis survived Holm correction ($p = 9.54 \cdot 10^{-7}$ each). Four-cell, per-task OLS summaries were 3.425 for B2 and 0.219 for B3 tokens per context token (conditional paired $p = 1.91 \cdot 10^{-6}$); they are not marginal derivatives across the compression boundary. These are system-package effects, not an isolated orchestration effect. Both architectures attained pass@1 = 1.0; with zero harmful outcomes among only 20 unique pairs, an iid-binomial planning reference gives a 13.91-point one-sided upper bound, so 2-point quality non-inferiority is not established. An exploratory small-model screen agrees perfectly with retained HumanEval outcomes but poorly with an approximate SWE-bench Lite protocol; the policy effect was not evaluated. The Pro pilot yields \$429.96 API-inference savings per 10,000 tasks; task-cluster resampling gives a \$417.33–\$442.90 stability range, not a population interval or full-TCO estimate. The evidence supports a long-context resource-efficiency mechanism

in the fixed task sets while leaving quality preservation, router effectiveness, component attribution, and real-repository transfer open.

Keywords: Large language models, agent systems, Introspection of Thought, token efficiency, inference economics, HumanEval, reproducible experiment

1 Introduction

A modern software-engineering agent must successively understand the task specification, retrieve the relevant context from a repository, and construct a change that can then be checked by tests or static analysis. In practice, these functions are often distributed across several roles: one role builds a plan, another proposes a solution, a third critiques the intermediate result, and a fourth runs external verification. This decomposition is conceptually convenient, but it becomes expensive when all roles are forced to reread the same long context.

This paper focuses precisely on that setting: tasks in which the code, documentation, compiler messages, and execution logs are shared across all roles. We are interested not only in the final solution quality but also in the cost of the chosen orchestration scheme in tokens, response time, and monetary spending. The article therefore compares classical external multi-agent organization with a hybrid scheme in which part of the role discussion is moved inside a single model call.

1.1 The problem of repeated context transmission and its practical significance

In agent systems built on large language models, a typical configuration is external multi-agent orchestration (Classical-MAS): separate processes or calls implement the planner, worker, critic, and tester roles [16–18]. In software-engineering tasks, where several roles read the same long fragments of code, logs, and documentation, this scheme has a structural defect: *each role receives the shared context $\mathcal{C}_0$ anew*, and additional coordination messages circulate between roles. In token-budget terms, this yields the term

$$D_{\text{ctx}} = \Theta(|\mathcal{A}| \cdot |\mathcal{C}_0|), \quad H_{\text{coord}} = O(|\mathcal{A}|^2 \cdot \bar{h}),$$

where $|\mathcal{A}|$ is the number of roles and $\bar{h}$ is the average coordination-message length. The coordination term is a topology-dependent all-pairs upper order, not a universal quadratic lower bound. In our API pilot, repeated input transmission was observed directly. At a target context of 16,384 tokens, Classical-MAS used 59,631 tokens per task on average, of which 57,485 (96.4%) were input tokens; Hybrid-INoT used 6,608 tokens. Per-case cost was \$0.14072 and \$0.02949, respectively, at the recorded Gemini 3.1 Pro Preview prices [35]. These values concern the first five HumanEval tasks and must not be generalized to a different task distribution without measurement.

The problem is therefore quantitatively observable: in the studied long-context regime, repeated input transmission produced an approximately ninefold difference in

total tokens between B2 and B3. The remainder of the paper separates the analytical model from empirical evidence and restricts claims to completed pilots.

1.2 Existing approaches and their limitations

The closest design space is broader than a self-reflection-versus-MAS dichotomy. It includes single-model refinement [7, 8], reasoning-action agents [6, 21], multi-persona prompting within one LLM [9], role collaboration through a shared blackboard [10], sparse external debate [11], and deliberately simple repository-repair pipelines such as Agentless [12]. Hybrid-INoT builds directly on INoT [1]. Its claimed novelty is therefore deliberately narrow: an operational integration of an internal planner-worker-critic call, deterministic context selection, external verification, an explicit deployment router, and auditable resource accounting. We do not claim invention of role prompting, self-critique, compression, or tool verification in isolation.

1.3 Main results and contribution

The paper delivers the following results:

- C1.** An operational definition of Hybrid-INoT with an explicit state signature, the `RouteMode` routing function, and a termination condition for the internal role cycle (§4).
- C2.** Proposition 2, a sufficient accounting condition for a token advantage of the complete Hybrid-INoT package under explicit bounds (§5).
- C3.** Inequality (3), which gives the profitability condition for a small self-check model (§5).
- C4.** A system of metrics for token efficiency, monetary cost, and code maintainability with a justified aggregation form (§6).
- C5.** An artifact-backed real-API evaluation covering context-length scaling, ablations, an exploratory small-model screening diagnostic, and E6 cross-model sensitivity (§7).
- C6.** Evidence-aligned hypothesis verdicts: the full conjunctive H1 is not established, H2 was not causally evaluated, and H3 was not supported in the direct architectural comparison (§7).

The harness, configurations, raw results, and figure-generation script are linked to an immutable research commit (§7).

2 Related work and distinction of the proposed approach

2.1 From chain-of-thought to internal role debate

Wei et al. [5] show that explicit verbalization of intermediate reasoning steps can improve solution quality on tasks that require several connected logical transitions. In this context, intermediate steps are textual fragments by which the model decomposes a complex inference into a sequence of local subproblems, while quality refers to the

final success of solving the target task. The Self-Refine approach [7] formalizes a “generator–feedback–revision” cycle inside a single model; Reflexion [8] supplements such a scheme with reflective memory, whereas ReAct [6] couples internal reasoning with external actions and tools. Sun and Zeng [1] describe *Introspection of Thought* (INoT) as a structured internal debate among several roles. At the same time, Turpin et al. [13] show that intermediate chain-of-thought text should not be interpreted as a faithful description of the hidden computational process. Accordingly, in the present paper introspection is treated primarily as a computational mechanism for candidate selection, critique, and selective rerun rather than as a literal window into the latent state of the model.

2.2 Multi-agent systems for software engineering

Weiss [14] and Wooldridge [15] provide the theoretical framework for MAS. MetaGPT [16] and ChatDev [17] transfer this framework into the context of large language models: roles are implemented as separate calls, each receiving its own copy of the context. Survey work identifies two key limitations of Classical-MAS: error propagation along the chain and the growth of coordination cost [18, 19]. SWE-bench and SWE-agent [20, 21] further show that practical software-engineering automation requires long context and iterative editing, which makes the retransmission problem D_{ctx} even more acute.

Single-Persona Prompting (SPP) simulates several personas through multi-turn prompting of one LLM [9]; Self-Collaboration Code Generation coordinates specialized roles through a shared blackboard and a linguistic tester [10]; S2-MAD reduces the communication graph of external debate [11]. Agentless, in contrast, demonstrates that a compact localization–repair–validation pipeline can be a strong software-engineering baseline without a complex agent society [12]. These works make component-level baselines and call topology essential to any novelty claim.

2.3 Context compression and external verification

LongLLMLingua and LLMLingua-2 [24, 25] demonstrate that prompt compression can preserve or even improve quality under long context while lowering cost. Jia et al. [26] apply this logic directly to code. For code self-check, linguistic reflection alone is insufficient: external observable signals are required, namely tests, static analysis, and compiler feedback [27–29].

2.4 Comparison of Hybrid-INoT with prior approaches

To systematize the closest approaches, we distinguish role mechanism, model-call topology, external verification, routing, and explicit resource accounting.

Table 1 shows substantial overlap at the component level. The contribution assessed here is the integrated and instrumented package, not categorical architectural precedence.

Table 1 Comparison of Hybrid-INoT with adjacent architectural classes

Method	Role mechanism / calls	External verification	Routing	Cost treatment
Self-Refine / Reflexion	Iterative self-feedback; repeated calls.	Optional / task dependent.	Fixed.	No architecture-level token accounting.
SPP	Multiple personas in one LLM, multi-turn.	Not central.	Fixed.	Call reduction is not the target estimand.
Self-Collaboration	Specialized roles, shared blackboard, multiple calls.	Linguistic tester; tool use is implementation dependent.	Fixed.	Efficiency reported, but no context-slope model.
MetaGPT / ChatDev / S2-MAD	External agents or sparse debate; multiple calls.	Pipeline dependent.	Fixed communication policy.	System-level cost.
Agentless	Deterministic repository pipeline; few model calls.	Patch validation.	Fixed stages.	Simplicity and repair cost are explicit baselines.
Hybrid-INoT	Planner-worker-critic prefixes in one call; one selected context.	Required after generation.	Tool/parallelism flags in the implementation.	Token balance, paired API cost, and context slope.

3 Formal problem statement

3.1 Task

Let $\mathcal{X}$ denote the set of textual task specifications, $\mathcal{C}$ the set of finite sequences of contextual artifacts (code files, documentation fragments, and execution logs), and $\mathcal{Y}$ the set of candidate solutions representable as textual artifacts and amenable to external verification. Let $(\Omega, \mathcal{F}, \mathbb{P})$ be the probability space induced by stochastic model generation, seed choice, and nondeterminism in external tool calls.

Definition 1 (Agentic engineering task). *An agentic engineering task is the tuple $(x, \mathcal{C}_0, \mathcal{Z}, \mathcal{V})$, where $x \in \mathcal{X}$ is a textual task description; $\mathcal{C}_0 \in \mathcal{C}$ is the initial task context, including code, documentation, and execution logs; $\mathcal{Z}$ is a finite set of permitted external tools; and $\mathcal{V} : \mathcal{Y} \rightarrow \{0, 1\}$ is an externally verifiable success function, for example, the execution of a test suite.*

The agent strategy induces a random output $Y : \Omega \rightarrow \mathcal{Y}$. At the same time, random variables $T : \Omega \rightarrow \mathbb{R}_{\geq 0}$, $C_{\text{inf}} : \Omega \rightarrow \mathbb{R}_{\geq 0}$, and $\ell : \Omega \rightarrow \mathbb{R}_{\geq 0}$ are defined, representing, respectively, the total token consumption, the monetary inference cost, and the latency of solving one task. The goal of the agent is to maximize $\mathbb{E}[\mathcal{V}(Y)]$ under the budget constraints $\mathbb{E}[T] \leq B$, $\mathbb{E}[C_{\text{inf}}] \leq B_{\text{\$}}$, and $\mathbb{E}[\ell] \leq B_{\ell}$.

Main notation

$\mathcal{C}_0$ — the initial shared task context; $|\mathcal{C}_0|$ is its token length.

$\hat{\mathcal{C}}$ — the compressed version of the initial context used in the internal cycle.
 $\mathcal{A}$ — the set of external roles in the Classical-MAS scheme; $|\mathcal{A}|$ is the number of such roles.
 Y — the random output of the agent with values in $\mathcal{Y}$; $y \in \mathcal{Y}$ is an individual candidate solution.
 K — the number of candidates generated by the role r_{work} in one iteration of the internal cycle.
 T — the random variable of total token consumption for one task; $\mathbb{E}[T]$ is taken with respect to the distribution defined on $(\Omega, \mathcal{F}, \mathbb{P})$.
 U_{tok} — the token-efficiency measure defined as the ratio of final solution quality to total token consumption.
 τ^* — a theoretical crossover implied only when the sufficient accounting assumptions hold; it is not identified by E3.
 δ_{H1} — the minimal practically relevant gain in U_{tok} used in Hypothesis H1.
 C_{check}^S — the cost of the preliminary check performed by the small model.
 C_{rerun}^L — the cost of rerunning the large model.
 ρ_0, ρ_1 — the probabilities of an expensive rerun without and with the preliminary check, respectively.

3.2 Research hypotheses

H1 (*advantage under long context*). There exists a threshold initial-context length τ^* such that for $|\mathcal{C}_0| > \tau^*$, Hybrid-INoT yields at least a $\delta_{H1} = 15\%$ increase in U_{tok} relative to Classical-MAS, while pass@1 degrades by no more than 2 percentage points.

Verification: E3 (§7.2) reports the finite-set mean and pairwise threshold counts; Wilcoxon tests, task-resampling ranges, and Holm correction are conditional sensitivity analyses rather than design-based population inference.

H2 (*economic justification of the small self-check model*). There exists a regime in which a small-model preliminary check reduces expensive reruns enough for $C_{\text{check}}^S < (\rho_0 - \rho_1)C_{\text{rerun}}^L$.

Intended verification: an integrated policy experiment recording per-case rerun decisions and costs. The retained E4 (§7.4) does not satisfy this design and is reported only as an exploratory screening diagnostic.

H3 (*boundary of applicability*). On tasks with genuinely independent subtasks and parallel tools, the response-time advantage of Hybrid-INoT vanishes or becomes negative.

Verification: an architectural comparison on five tasks and a separate controlled serial/parallel tool microbenchmark in E2c (§7.3).

When testing H1, model, tools, verification, task, seed, temperature, and task prompt are matched within each pair. The treatment is nevertheless a *system bundle*: B2 uses three calls over uncompressed context, while B3 combines the role prefixes in one call and applies deterministic selection above 4,096 tokens. Thus E3 identifies the package effect, not the isolated causal contribution of orchestration or compression. The threshold $\delta_{H1} = 15\%$ is treated as the minimum practically meaningful effect; observed resource savings do not replace evaluation of the full conjunctive H1 criterion.

4 Method: Hybrid-INoT

4.1 States and role signatures

Define the system state as $\sigma = (x, \hat{\mathcal{C}}, \mathcal{H}, i)$, where $\hat{\mathcal{C}}$ is the compressed context, $\mathcal{H}$ is the history of iterations, and $i \in \{0, \dots, I\}$ is the iteration counter. The evaluated client maintains a separate experiment-level monetary counter and rejects a request whose estimated charge would cross the configured hard cap.

Let Π be the space of plans, and let $K \in \mathbb{N}$ denote the number of candidates generated by the role r_{work} in one iteration. Then each role is defined by a signature $r : \mathcal{S}_r^{\text{in}} \rightarrow \mathcal{S}_r^{\text{out}}$:

$$\begin{aligned} r_{\text{plan}} &: (x, \hat{\mathcal{C}}) \mapsto \pi \in \Pi, \\ r_{\text{work}} &: (\pi, \hat{\mathcal{C}}, K) \mapsto \mathbf{y} = (y_1, \dots, y_K) \in \mathcal{Y}^K, \\ r_{\text{crit}} &: (\mathbf{y}, \hat{\mathcal{C}}) \mapsto (s_1, \dots, s_K) \in [0, 1]^K, \\ r_{\text{self}} &: (y \in \mathcal{Y}) \mapsto (v, M, e) \in \{0, 1\} \times [0, 1] \times \mathcal{E}. \end{aligned}$$

The roles $r_{\text{plan}}, r_{\text{work}}, r_{\text{crit}}$ are implemented as role prefixes inside *one* large-model call over the shared $\hat{\mathcal{C}}$; the role r_{self} is an external process running tests and static analysis.

4.2 Routing function

Definition 2 (RouteMode). *Let $\varphi_{\text{tool}}(x) \in \{0, 1\}$ read the Boolean `requires_tool` task-metadata field and let $\varphi_{\text{par}}(x) \in \{0, 1\}$ indicate whether the metadata field `parallelism` is at least two. The implemented rule at the pinned research commit is*

$$\text{RouteMode}(x) = \begin{cases} \text{external}, & \varphi_{\text{tool}}(x) = 1 \vee \varphi_{\text{par}}(x) = 1, \\ \text{internal}, & \text{otherwise.} \end{cases}$$

The pinned code does not infer these flags from arbitrary task instructions: the benchmark loader supplies them for the synthetic tool tasks. A learned or instruction-based metadata classifier and a context-length gate are possible deployment extensions, but neither is present in the evaluated commit. E3 forces the internal B3 treatment at every context cell and therefore evaluates neither the router nor any routing threshold.

4.3 Internal cycle and termination condition

Proposition 1 (Termination of the internal cycle). *Assume that each operation within one iteration of Algorithm 1 terminates in finite time and that the number of iterations is bounded above by the parameter I . Then the algorithm terminates no later than after I iterations: it either returns a solution after a successful external check, stops after iteration $i > 1$ when $s_{i,k^*} - s^{\text{prev}} < \varepsilon$, or, after exhausting the iteration limit, returns the best candidate found.*

Algorithm 1 Hybrid-INoT: internal introspective cycle

Require: task x , context $\mathcal{C}_0$, maximum iterations I , candidates K , rerun limit R **Ensure:** solution y^* , flag v^* , score M^*

```
1:  $\mathcal{C} \leftarrow \text{PolicySanitize}(x, \mathcal{C}_0)$ 
2:  $\hat{\mathcal{C}} \leftarrow \text{CompressIfNeeded}(\mathcal{C})$  ▷ Alg. 2
3:  $m \leftarrow \text{RouteMode}(x)$  ▷ Def. 2
4: if  $m = \text{external}$  then return  $\text{ExternalAgentWorkflow}(x, \hat{\mathcal{C}})$ 
5: end if
6:  $s^{\text{best}} \leftarrow -\infty$ ;  $s^{\text{prev}} \leftarrow -\infty$ ;  $y^{\text{best}} \leftarrow \perp$ ;  $r \leftarrow 0$ ;  $e \leftarrow \emptyset$ 
7: for  $i = 1$  to  $I$  do
8:    $(\pi_i, \{y_{i,k}\}, \{s_{i,k}\}) \leftarrow \text{CombinedLargeModelCall}(x, \hat{\mathcal{C}}, K, e)$  ▷ one API call
   with planner/worker/critic prefixes
9:    $k^* \leftarrow \arg \max_k s_{i,k}$ ;  $y_i \leftarrow y_{i,k^*}$ 
10:  if  $s_{i,k^*} > s^{\text{best}}$  then  $s^{\text{best}} \leftarrow s_{i,k^*}$ ;  $y^{\text{best}} \leftarrow y_i$ 
11:  end if
12:   $(v_i, M_i, e_i) \leftarrow r_{\text{self}}(y_i)$  ▷  $e_i$ : verified tool output
13:  if  $v_i = 1$  then return  $(y_i, 1, M_i)$ 
14:  end if
15:   $e \leftarrow \text{Truncate}(e_i, 600)$ 
16:  if  $r < R$  then
17:     $(y'_i, v'_i, M'_i, e'_i) \leftarrow \text{SelectiveRerun}(x, \{y_{i,k}\}, \{s_{i,k}\})$ ;  $r \leftarrow r + 1$ 
18:    if  $v'_i = 1$  then return  $(y'_i, 1, M'_i)$ 
19:    end if
20:  end if
21:  if  $i > 1 \wedge s_{i,k^*} - s^{\text{prev}} < \varepsilon$  then break ▷ see Prop. 1
22:  end if
23:   $s^{\text{prev}} \leftarrow s_{i,k^*}$ 
24: end for
25: return  $(y^{\text{best}}, 0, M(y^{\text{best}}))$ 
```

Proof The outer loop over the index i contains at most I passes. Each iteration consists of a finite sequence of steps: plan construction, generation of K candidates, critical scoring, external verification, and context update. By assumption, all of these operations terminate in finite time; therefore, each iteration also terminates. Consequently, the algorithm either stops early by the instructions **return** or **break**, or reaches the final **return** after the I -th iteration. Hence the running time of the internal cycle is bounded above by a finite number of iterations, which proves the statement. $\square$

Note

Proposition 1 guarantees only algorithm termination; it does not imply monotonic growth of the scores s_i , nor does it guarantee the correctness of the critic itself. The scenario in which the critic systematically endorses incorrect solutions is examined separately in E2b (§7.3) as one of the threats to validity.

4.4 Context compression

Algorithm 2 Compression of long context

Require: $\mathcal{C}_0$, length threshold τ_{len}

Ensure: $\hat{\mathcal{C}}$

- 1: **if** $|\mathcal{C}_0| \leq \tau_{\text{len}}$ **then return** $\mathcal{C}_0$
 - 2: **end if**
 - 3: split $\mathcal{C}_0$ into text blocks $\{b_j\}$ and extract task identifiers
 - 4: $\text{rel}(b_j) \leftarrow 5 r_{\text{id}}(b_j) + 2 \mathbb{I}_{\text{sig}}(b_j) + 4 \mathbb{I}_{\text{err}}(b_j) - 0.001 |b_j|$
 - 5: retain signature/error blocks; then add remaining blocks by descending $\text{rel}(b_j)$
 - 6: **return** blocks whose summed proxy counts fit $\tau_{\text{len}} = 4096$
-

Here r_{id} is the fraction of task identifiers present in a block, while $\mathbb{I}_{\text{sig}}$ and $\mathbb{I}_{\text{err}}$ mark signatures and error/log text. The implementation budgets the sum of block proxy counts but joins retained blocks with separators afterward; consequently, the serialized context can slightly exceed 4,096 proxy tokens (the observed E3 maximum is 4,170). This deterministic lexical proxy is inspired by learned prompt compression [24, 25], but it is not an LLMingua-2 classifier. Stating the deployed score and this boundary behavior makes the treatment reproducible and prevents attributing its effect to a model that was not run.

4.5 Selective rerun

Algorithm 3 Selective rerun based on verified signals

Require: task x , candidates $\{y_1, \dots, y_K\}$, scores $\{s_k\}$

Ensure: (y^*, v^*, M^*, e^*)

- 1: sort $\{y_k\}$ in descending order of s_k ; keep the top $M \leq K$
 - 2: **for** each y_m **do**
 - 3: $(v_m, M_m, e_m) \leftarrow r_{\text{self}}(y_m)$
 - 4: **if** $v_m = 1$ **then return** $(y_m, 1, M_m, e_m)$
 - 5: **end if**
 - 6: **end for**
 - 7: construct one correction prompt from x , the top candidate, and $\text{Truncate}(e_m, 600)$
 - 8: $y' \leftarrow \text{LargeModelRerun}(\text{prompt})$
 - 9: $(v', M', e') \leftarrow r_{\text{self}}(y')$
 - 10: **return** (y', v', M', e')
-

5 Analytical model of token and monetary costs

5.1 Structure of the analytical relations

The analysis in this section includes two types of relations. The first comprises direct balance equalities and inequalities that define the structure of token and monetary costs. The second comprises statements comparing architectures under fixed assumptions about context length, coordination cost, and internal critique. This separation makes it possible to distinguish general accounting relations from results that require additional assumptions.

5.2 Token budgets of Classical-MAS and Hybrid-INoT

Assumption 1. *In each of I iterations of Classical-MAS, at least $\alpha|\mathcal{A}|$ roles, with $\alpha \in (0, 1]$, read the shared context $\mathcal{C}_0$ in full. Hybrid-INoT uses context $\hat{\mathcal{C}}$ satisfying $|\hat{\mathcal{C}}| \leq q_{\max}|\mathcal{C}_0|$ for a fixed $q_{\max} \leq 1$.*

Assumption 2. *For coefficients $\gamma, \beta \geq 0$, the one-off compression cost satisfies $T_{\text{cmp}} \leq \gamma|\mathcal{C}_0|$, the additional internal-role cost satisfies $T_{\text{core}}^{\text{add}} \leq \beta|\hat{\mathcal{C}}|$, and the absolute difference between all remaining generation, verification, and rerun terms of the two systems is bounded by a constant K_0 independent of $|\mathcal{C}_0|$.*

Proposition 2 (Sufficient accounting condition). *Let*

$$A_0 = I\alpha|\mathcal{A}| - (I + \beta)q_{\max} - \gamma.$$

Under Assumptions 1–2, if $A_0 > 0$, then

$$\mathbb{E}[T_{\text{MAS}} - T_{\text{INoT}}] \geq A_0|\mathcal{C}_0| - K_0,$$

and the expected token difference is positive for $|\mathcal{C}_0| > K_0/A_0$.

Accounting derivation Assumption 1 gives the context-transmission lower bound

$$T_{\text{ctx}}^{\text{MAS}} \geq I \cdot \alpha|\mathcal{A}| \cdot |\mathcal{C}_0|.$$

For Hybrid-INoT, the repeated selected-context term, internal-role overhead, and compression together are bounded by

$$(I + \beta)|\hat{\mathcal{C}}| + \gamma|\mathcal{C}_0| \leq ((I + \beta)q_{\max} + \gamma)|\mathcal{C}_0|.$$

Subtracting these terms and applying the K_0 bound to all residual differences yields $A_0|\mathcal{C}_0| - K_0$. If $A_0 > 0$, this lower bound is positive exactly when $|\mathcal{C}_0| > K_0/A_0$. $\square$

Scope of applicability

Proposition 2 is a sufficient accounting result, not a causal theorem about orchestration and not an empirical validation of a router. Coordination traffic, if nonnegative, can only strengthen the bound but is not used as an unsupported asymptotic lower bound. E3 estimates slopes of the evaluated B2/B3 packages; because no crossover is bracketed, it cannot identify K_0/A_0 or a deployment threshold.

5.3 Request-count identity and output cap

The implementation produces all K candidates, the plan, and the critique in one core call per iteration; multiplying a per-candidate prompt by K would therefore double-count the shared prompt. Let $N_{\text{core}} \leq I$ and $N_{\text{rerun}} \leq \min(I, R)$ be the realized numbers of core and rerun calls, and let P and O denote the provider-recorded input and output tokens of an individual call. For the internal route, model-token accounting is the identity

$$T_{\text{INoT}}^{\text{internal}} = \sum_{i=1}^{N_{\text{core}}} (P_{\text{core},i} + O_{\text{core},i}) + \sum_{j=1}^{N_{\text{rerun}}} (P_{\text{rerun},j} + O_{\text{rerun},j}). \quad (1)$$

The pinned implementation sets provider output caps $O_{\text{core},i} \leq 2048 + 700K$ and $O_{\text{rerun},j} \leq 1024$, hence

$$T_{\text{out}} \leq N_{\text{core}}(2048 + 700K) + 1024N_{\text{rerun}}.$$

Compression is deterministic and incurs no model call. Its block-sum budget is 4,096 proxy tokens, but join separators are excluded from that counter, so the serialized E3 context reaches at most 4,170 proxy tokens. No corresponding hard bound exists for total input tokens because the task prompt is not capped and proxy counts need not equal provider tokenization. The evaluated code enforces a pre-request monetary cap at the shared API client, but not a per-task token counter. We therefore report the exact recorded identity and output-only cap, not a hard total-token guarantee.

5.4 Profitability condition for the small self-check model

Inference cost:

$$C_{\text{inf}} = c_L^{\text{in}} \sum_i p_i^L + c_L^{\text{out}} \sum_i r_i^L + c_S^{\text{in}} \sum_i p_i^S + c_S^{\text{out}} \sum_i r_i^S. \quad (2)$$

Under a two-scenario accounting model with exactly one preliminary check, at most one avoided rerun, and a constant rerun cost, a small self-check model is economically beneficial if and only if

$$C_{\text{check}}^S < (\rho_0 - \rho_1) \cdot C_{\text{rerun}}^L. \quad (3)$$

The left-hand side is the cost of the preliminary check, whereas the right-hand side is the expected savings *if an integrated policy actually prevents some large-model reruns*. Inequality (3) is therefore a prospective design condition. The retained E4 harness records only aggregate screening agreement and does not implement or identify the policy effect $\rho_0 - \rho_1$.

6 Metric system

6.1 Main metrics and the rationale for the aggregation form

$$U_{\text{tok}} = \frac{Q}{T}, \quad Q_s = \frac{Q}{C_{\text{inf}}}, \quad \text{VCR} = \frac{1}{N} \sum_i \mathbb{I}[\text{tests}_i = 1 \wedge \text{static}_i = 1 \wedge \text{sec}_i = 1]. \quad (4)$$

For heterogeneous quality, the paper uses a weighted sum over task classes, $Q = \sum_b w_b Q_b$, with weights specified in Table 2.

6.2 Aggregated success metric and its form

The aggregated success metric is defined as

$$\text{MAS}_i = \min(\text{Success}_i, \text{Verified}_i) \cdot M_i^\lambda, \quad \lambda \in [0, 1], \quad \lambda = 0.5. \quad (5)$$

The rationale for this construction is as follows. First, Success and Verified are necessary conditions for counting a solution as successful: a patch that fails external verification should not receive a high final score. Second, the multiplier M_i^λ models the effect of maintainability on the final assessment; the parameter λ controls the strength of this effect. The baseline uses $\lambda = 0.5$; E2d produced seven ranking inversions under weight and λ changes, so the composite metric is treated as secondary.

6.3 Maintainability weights and their rationale

$$M_i = w_{\text{cc}}(1 - \widehat{\text{CC}}_i) + w_{\text{dup}}(1 - \widehat{\text{Dup}}_i) + w_{\text{warn}}(1 - \widehat{\text{Warn}}_i) + w_{\text{doc}}\widehat{\text{Doc}}_i. \quad (6)$$

Table 2 Maintainability weights and their relation to ISO/IEC 25010:2023 subcategories [30]

Parameter	Value	Rationale
w_{cc}	0.35	ISO 25010 Modifiability; cyclomatic complexity [31] is the most direct proxy for maintenance cost.
w_{dup}	0.25	ISO 25010 Reusability; duplication is a key signal of technical debt.
w_{warn}	0.20	ISO 25010 Analysability; warnings are a surface-level yet reliable signal.
w_{doc}	0.20	ISO 25010 Modifiability; documentation coverage is used as a proxy for comprehensibility.

Sensitivity analysis

E2d tested the robustness of the configuration ranking under $(w_{\text{cc}}, w_{\text{dup}}, w_{\text{warn}}, w_{\text{doc}}) \pm 20\%$ and $\lambda \in \{0, 0.25, 0.5, 0.75, 1\}$ in (5); seven reversals were observed.

6.4 Reliability metrics

$$\text{SCR} = \frac{\#\{\text{incorrect candidates detected by the self-check procedure}\}}{\#\{\text{all incorrect candidates}\}}, \quad (7)$$

$$\text{RFR} = \frac{\#\{\text{patches without rollback}\}}{\#\{\text{applied patches}\}}. \quad (8)$$

7 Empirical validation

7.1 Reproducibility, source data, and statistical protocol

The research harness is published in the INoT_Research repository [34]; every result below is tied to commit 010211ee5775185e8330ab6cde06e912c2adcb9e. The built-in configuration check completed successfully, and the test suite reported 15 passing tests. Raw JSON inputs are stored under `results/slide_e2e3_real`, `results/e6`, and `reproducibility/results/20260726_flash_lite`. Their portable relative paths, SHA-256 hashes, aggregates, and figure-generation code are captured in `reproducibility/evidence_snapshot.json`. Grid checks found 60/60 expected Pro rows and 240/240 Flash-Lite rows, with no duplicate keys, missing cells, token-accounting mismatches, or usage-trace mismatches. The unit of analysis is task $\times$ architecture $\times$ target context $\times$ seed.

The primary dataset is canonical HumanEval [22]. The Pro pilot uses HumanEval/0–4, and the Flash-Lite replication uses HumanEval/0–19 in canonical order. Historical context is generated deterministically (seed 42) from prompts and canonical solutions of *other* HumanEval tasks; the target task solution is excluded. Whole blocks are appended, so actual lengths exceed targets. For $n = 20$ and targets $\{256, 1024, 4096, 16384\}$, mean actual lengths are $\{371.2, 1154.8, 4210.6, 16524.3\}$ tokens, with ranges $\{259\text{--}545, 1024\text{--}1411, 4100\text{--}4415, 16389\text{--}16946\}$. This context is reproducible but synthetic.

B2 Classical-MAS invokes planner, worker, and critic separately over the uncompressed shared context. B3 combines those role instructions in one call and applies Algorithm 2 only above 4,096 tokens; external verification follows generation. Thus the primary estimand is an end-to-end package contrast, not an isolated orchestration effect. E3 runs B3 in internal mode throughout, so it does not evaluate `RouteMode`. Models are `google/gemini-3.1-pro-preview` and `google/gemini-3.1-flash-lite`. Recorded prices are \$2/\$12 and \$0.25/\$1.50 per million input/output tokens, respectively [35, 36], fixed on 26 July 2026. These API aliases are not immutable weight snapshots, so provider drift remains possible.

For each context length, B2 and B3 are paired on task and seed. We report

$$S_T = 100 \left(1 - \frac{\bar{T}_{B3}}{\bar{T}_{B2}} \right), \quad S_C = 100 \left(1 - \frac{\bar{C}_{B3}}{\bar{C}_{B2}} \right), \quad (9)$$

$$\Delta U_{\text{tok}} = 100 \left(\frac{U_{\text{tok}}^{B3}}{U_{\text{tok}}^{B2}} - 1 \right). \quad (10)$$

Means, minima, and pairwise threshold counts are descriptive finite-set estimands. Wilcoxon signed-rank tests [32] are reported only as conditional sensitivity analyses of a symmetric location shift; they do not test the reported mean. The 15% analysis applies the one-sided signed-rank test to $\Delta U_{\text{tok}} - 15\%$, and each four-context family is adjusted by Holm’s sequential procedure [33]. Percentile ranges from 10^4 task-level resamples describe stability to reweighting the observed tasks, not population confidence coverage. Binary quality is summarized by harmful discordance (B2 pass/B3 fail); the zero-event Clopper–Pearson calculation is an iid-binomial planning reference, not design-valid inference for the first non-random HumanEval tasks. The four repeated context cells are not treated as independent quality observations, and the single retained API run does not measure seed or provider-run variance.

7.2 E3 — Scaling with context length

Table 3 reports observed API means. B2 grows with repeated context, whereas B3 remains around 5.6–6.7k tokens after the 4,096-token target because of compression.

Table 3 Real API measurements for B2 and B3 on HumanEval

Model	Context	T_{B2}	T_{B3}	S_T , %	S_C , %	pass@1 B2/B3
Pro ($n = 5$)	256	4964	2203	55.6	43.1	1.00/1.00
	1024	7117	2925	58.9	46.8	1.00/1.00
	4096	18218	6706	63.2	49.1	1.00/1.00
	16384	59631	6608	88.9	79.0	1.00/1.00
Flash-Lite ($n = 20$)	256	2738	1379	49.6	26.0	1.00/1.00
	1024	5487	2271	58.6	41.8	1.00/1.00
	4096	16007	5664	64.6	56.9	1.00/1.00
	16384	58110	5633	90.3	86.9	1.00/1.00

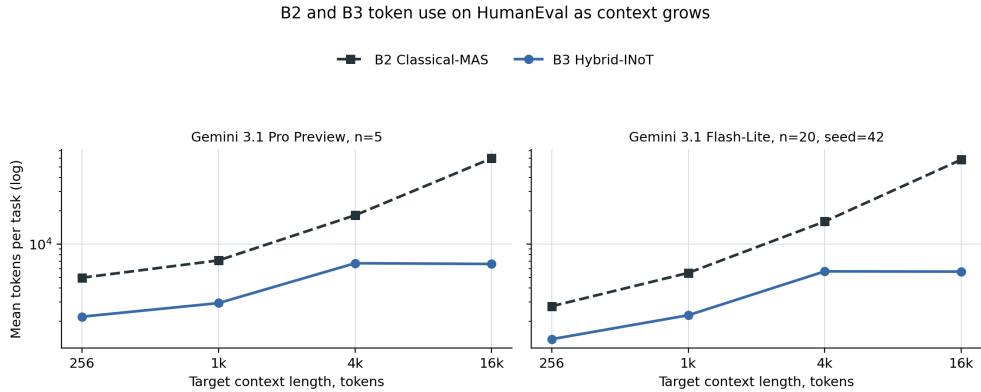


Figure 1 Mean total tokens per task. Dashed squares denote B2 Classical-MAS; solid circles denote B3 Hybrid-INoT. The y axis is logarithmic. Left: retained Pro pilot ($n = 5$ per point); right: Flash-Lite replication ($n = 20$, seed 42).

For Pro at 16,384 target tokens,

$$S_T = 100 \left(1 - \frac{6608.4}{59631.4} \right) = 88.9\%, \quad S_C = 100 \left(1 - \frac{0.0294908}{0.1407228} \right) = 79.0\%.$$

Since pass@1 is one, $U_{B2} = 1/59631.4 = 1.677 \cdot 10^{-5}$ and $U_{B3} = 1/6608.4 = 1.513 \cdot 10^{-4}$, an aggregate ratio-of-means gain of 802.4%. The primary paired estimand instead computes the relative ratio within each task before averaging: 805.6%, with a 756.0–850.4% task-resampling stability range.

All Pro paired gains exceed 15%, but with five one-directional pairs the conditional one-sided signed-rank $p = 0.03125$ at each context does not survive four-test Holm correction. In the Flash-Lite run, finite-set mean within-task gains are 106.5% (task-resampling range 87.5–126.1), 144.8% (131.9–158.3), 183.3% (177.3–189.6), and 934.6% (910.8–959.6). All 20 task pairs exceed 15% in every cell; the conditional one-sided signed-rank $p = 9.54 \cdot 10^{-7}$ survives Holm correction throughout. This supports a within-sample resource statement, not a population mean claim.

The scaling diagnostic fits, separately for each task, total tokens against *actual* context tokens over the four cells. Flash-Lite mean four-point OLS slopes are 3.425 (task-resampling range 3.421–3.430) for B2 and 0.219 (0.216–0.222) for B3 tokens per context token; every paired difference is positive (mean 3.206, range 3.201–3.212; conditional signed-rank $p = 1.91 \cdot 10^{-6}$). Pro summaries are 3.410 versus 0.234, with conditional $p = 0.0625$ (Fig. 2). Because B3 crosses the 4,096-token compression boundary, these coefficients summarize the complete four-cell grid; they are neither local marginal derivatives nor uncertainty estimates over seeds or repeated API calls.

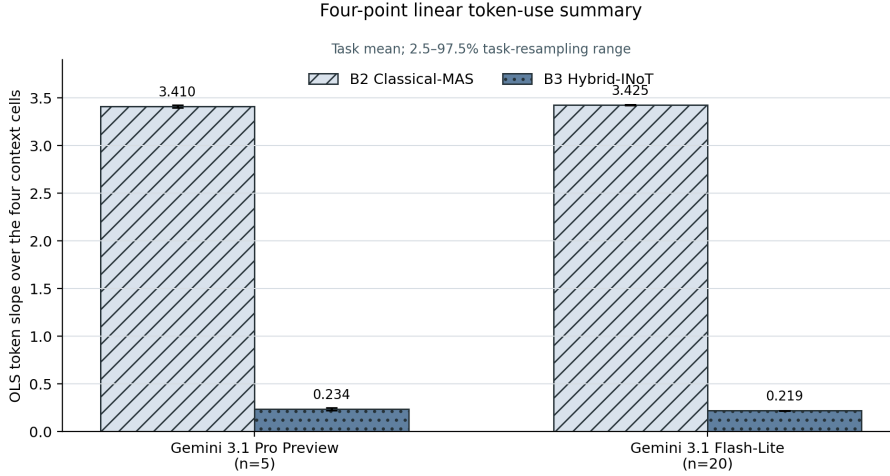


Figure 2 Per-task four-cell OLS summaries of total tokens against actual context length. Bars show task means and error bars show 2.5–97.5% task-resampling stability ranges. The estimand describes the complete B2/B3 packages and is not a marginal derivative across the compression boundary.

Pass@1 = 1.0 for both architectures creates a ceiling. Among 20 unique Flash-Lite task pairs there are zero harmful outcomes. Under an iid Bernoulli task-sampling model, the one-sided 95% zero-event reference bound is 13.91 percentage points; at least 149 zero-harm unique pairs would be required to push it below 2 points, and all 164 HumanEval tasks would yield 1.81 points if no harm were observed. Because the evaluated tasks are the first 20 rather than a random sample, these nominal bounds are planning calculations, not population-coverage guarantees. Thus the resource sub-criterion is met descriptively in the fixed sample, but the full conjunctive H1 is not established. No crossover is bracketed by E3: behavior below 256 target tokens is unknown, and neither a theoretical τ^* nor a routing threshold is identified.

7.3 E2 — Ablations, critic behavior, and parallelism

The Pro ablation pilot contains five tasks. pass@1 is 1.0 for full B3 and variants without compression, critic, or self-check. Paired aggregate-success differences for full B3 against the three variants are -0.0037 , -0.0262 , and 0.0000 , with $p = 1.0$, 0.5 , and 1.0 ; none survives Holm correction. Component token-overhead shares are 36.6% for compression, 29.7% for the critic, and 33.8% for self-check, but these are overhead allocations, not demonstrated independent quality contributions. Only two B0 failures were available for E2b; zero erroneous critic approvals in two cases do not establish critic reliability.

In E2c, mean B2 latency is 19.28 s and B3 latency 9.76 s, a -49.4% change; the task-resampling range for B3–B2 is $[-13.05, -6.83]$ s, while the conditional Wilcoxon test gives $p = 0.0625$ at $n = 5$. A separate deterministic tool microbenchmark shows serial execution to be 148.8% slower than parallel execution, yet the direct architectural comparison does not show the predicted B3 penalty. H3 is therefore *not supported*; the microbenchmark only confirms a mechanism requiring isolation on real independent-tool tasks. E2d produces seven ranking inversions under weight and λ changes, with the no-critic variant ranked first canonically. The composite metric is therefore unstable and retained only as a secondary measure.

7.4 E4 — Exploratory small-screen agreement diagnostic

Twenty HumanEval cases and twenty approximate SWE-bench Lite cases used Gemini 3.1 Pro Preview for generation and Flash-Lite for a PASS/REJECT screen. The retained file contains aggregate counts but not per-case decisions, candidate texts, costs, or counterfactual rerun outcomes. Rejecting a failed candidate in this harness does not itself avoid an implemented rerun. Consequently, the original policy estimand in Eq. (3) and H2 is not reproducible or causally evaluated; only agreement with the retained external-verification outcome can be reported.

HumanEval agreement is 1.00 (Clopper–Pearson 95% reference interval 0.832–1.000), with sensitivity, specificity, and balanced accuracy all 1.00; the mean screen cost is 0.555% of generation cost. Approximate SWE-bench agreement is 0.30 (reference interval 0.119–0.543), sensitivity 0.60, specificity 0.20, and balanced accuracy 0.40; screen cost is 0.846% of generation. The nominal intervals presume iid cases and are descriptive references for these fixed case sets. The asterisk is essential: the

Table 4 Aggregate screening diagnostic

Dataset	n	Correct/pass	Correct/reject	Failed/pass	Failed/reject
HumanEval	20	13	0	0	7
SWE-bench Lite*	20	3	2	12	3

loader does not run the official containerized SWE-bench harness [20]; it uses code extraction and a `check(_)` stub. These aggregates motivate a future integrated policy experiment but do not support H2.

7.5 E6 — Cross-model sensitivity

The retained E6 compares Pro and Flash-Lite on ten controlled cases per architecture–context cell. Three simple function families are repeated (`reverse_words`, `count_peaks`, and `normalize_path`); pass@1 is 1.0 throughout. At contexts 512, 2,048, and 8,192, Flash-Lite cost is 4.8, 6.9, and 9.7% of Pro cost for B2, and 4.4, 5.4, and 7.0% for B3 (Fig. 3). Paired U_{tok} differences are significant ($p = 0.001953$, all survive Holm correction), but the quality ceiling prevents generalization to difficult tasks.

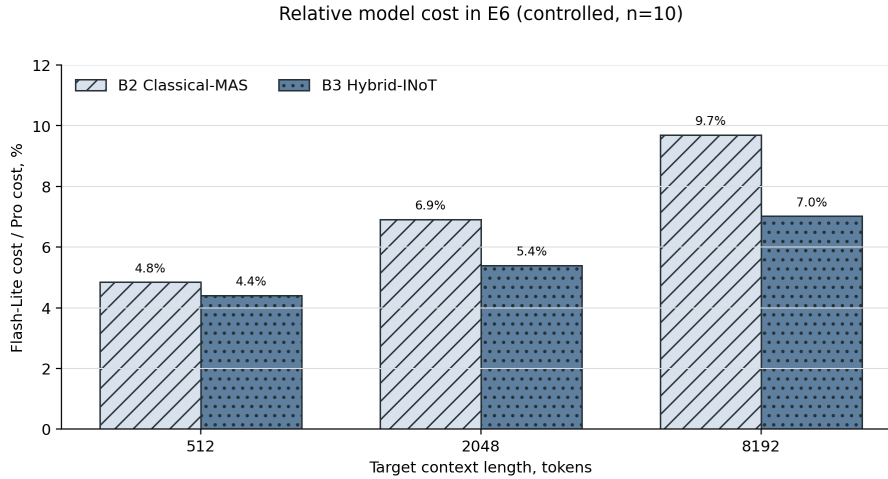


Figure 3 Mean Gemini 3.1 Flash-Lite cost relative to Gemini 3.1 Pro Preview in controlled E6. Hatching preserves black-and-white readability.

7.6 E5 — API-inference cost extrapolation

The full TCO experiment was not completed: an additional run hit the provider limit before `runs.json` was persisted and is excluded. Instead, we perform an explicitly

limited analytical extrapolation from twenty paired Pro E3 cases: five tasks at each of four context lengths with equal weights. For N tasks,

$$\Delta C_{\text{inf}}(N) = N \cdot \frac{1}{20} \sum_{i=1}^{20} (C_{B2,i} - C_{B3,i}). \quad (11)$$

The mean difference is \$0.042996 per task; at $N = 10,000$, B3 savings are \$429.96. Because the same five tasks recur at all four context lengths, resampling five *task clusters* while retaining all four cells gives a \$417.33–\$442.90 stability range. This range has no design-based population coverage target. Leave-one-task-out estimates range from \$425.20 to \$435.51. Mean input and output reductions are 17,146.8 and 725.2 tokens per case, and all 20 paired deltas are positive. Independently varying input and output price multipliers from 0.5 to 1.5 leaves estimated savings positive throughout (\$214.98–\$644.94; Fig. 4).

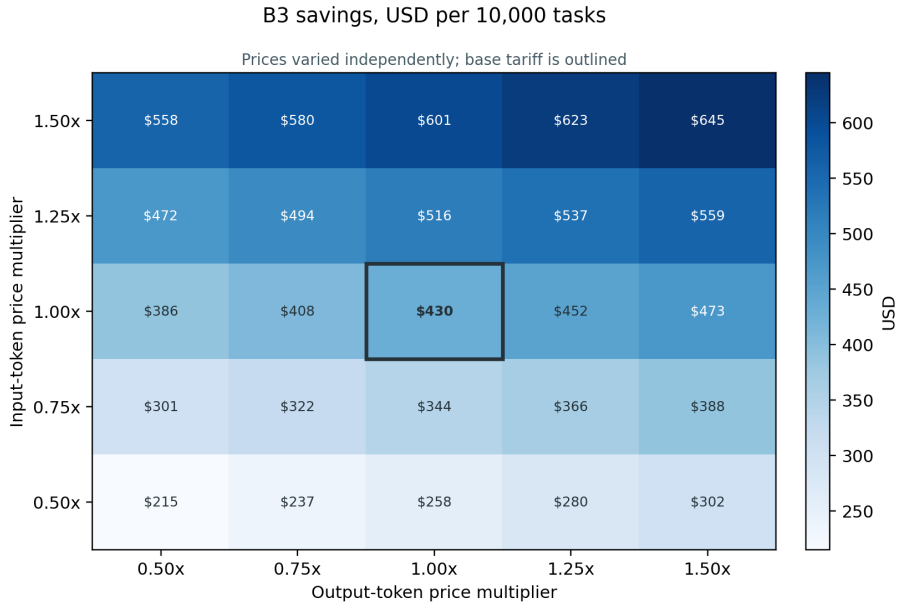


Figure 4 API-inference savings per 10,000 tasks under independent input/output price multipliers. The outlined cell is the recorded Pro tariff. This is deterministic price sensitivity, not population uncertainty.

8 Economic model

8.1 Boundary between inference cost and TCO

$$\text{TCO}_{\text{month}} = C_{\text{inf}} + C_{\text{hw}} + C_{\text{energy}} + C_{\text{storage}} + C_{\text{ops}}. \quad (12)$$

Only external-API C_{inf} is measured. Hardware, energy, storage, operations, engineering labor, and delay costs are not observed. Thus, E5 supports API-inference savings for the pilot mixture but not $\Delta\text{TCO} < 0$.

8.2 Hypothesis verdicts

Table 5 Original hypotheses versus observed evidence

Claim	Verdict	Evidence
H1	not established as a whole	Every fixed-sample Flash-Lite pair exceeds the 15% resource threshold, but the 2-pp quality non-inferiority subcriterion is not established.
H2	not evaluated	E4 retained screening aggregates, not an integrated rerun policy or per-case counterfactual costs.
H3	not supported	The tool microbenchmark favors parallelism, but direct B3 is faster than B2; $n = 5$.
τ^* /router	not identified	No crossover is bracketed, behavior below the grid is unknown, and E3 forces internal B3.

9 Limitations and threats to validity

1. **Small, non-random samples.** The first 5 or 20 HumanEval tasks are used rather than a random stratified sample of 164; the new run has one seed.
2. **Synthetic context.** Long inputs are assembled from other HumanEval tasks rather than repository history. They isolate retransmission cost but not real codebase dependencies.
3. **Quality ceiling.** B2/B3 and controlled E6 have $\text{pass}@1 = 1.0$, supporting a resource comparison but not a strong quality test.
4. **Bundled treatment.** B3 changes call topology and, above 4,096 tokens, context selection. Component attribution requires a factorial design; the present estimand is the end-to-end package.
5. **Approximate SWE-bench.** The official container harness is not used, so no claim about real issue resolution is made.
6. **Router not evaluated.** E3 always invokes internal B3; neither flag accuracy nor a context gate is tested.
7. **Post-hoc robustness analyses.** Four-cell OLS summaries, iid-reference harm bounds, and the corrected task-cluster resampling range were added after auditing the pilot. They are transparent diagnostics, not preregistered confirmatory endpoints.
8. **E4 auditability.** Per-case screen decisions and counterfactual rerun outcomes were not persisted, so H2 remains unevaluated.
9. **API drift.** Model names, parameters, usage, latency, and cost are recorded, but aliases do not guarantee immutable weights. Provider-limit failures are excluded.
10. **Limited answer auditability.** `runs.json` records metrics and usage but not complete final solution text.

11. **Cost is not TCO.** Figure 4 covers API inference only; the other terms in (12) are unmeasured.
12. **Architectural boundaries.** Independent subtasks, process isolation, critic errors, and compression losses may eliminate the B3 advantage. Role text is a computational mechanism, not a faithful explanation of hidden reasoning [13].

10 Directions for further research

The next study should remove the identified uncertainty rather than repeat the pilot:

1. preregister a full 2×2 factorial experiment crossing call topology (separate/combined) with context selection (off/on);
2. run at least 149 unique paired tasks with zero harmful outcomes—preferably all 164 HumanEval tasks and several seeds—for a 2-point exact non-inferiority target;
3. extend the context grid below 256 tokens and include both winning and losing regions before estimating any crossover or routing gate;
4. retain full input/output, prompt hashes, immutable model identifiers, and external-test results per run;
5. replace the approximate SWE-bench path with the official container harness on a random repository sample;
6. repeat E2c with real independent tools under matched models and serial/parallel execution;
7. compare B3 with Self-Refine, SPP, Self-Collaboration, S2-MAD, Agentless, ReAct, and SWE-agent under matched calls, context, and verification;
8. implement E4 as a randomized or paired policy experiment that persists every screen decision, rerun, final candidate, and cost;
9. measure full TCO, including operations, reruns, latency, infrastructure, and storage, under a production context-length distribution.

11 Conclusion

Hybrid-INoT is evaluated as an integrated intermediate system between in-context role prompting and external multi-agent orchestration. Across two real-API pilots, the B3 package used 49.6–90.3% fewer tokens than the matched B2 package; the Flash-Lite four-cell OLS summary is 3.425 for B2 and 0.219 for B3 tokens per context token. Every observed Flash-Lite pair exceeds the 15% resource threshold, and the conditional rank-based sensitivity analysis survives multiplicity correction, but the full H1 is not established: 20 ceiling-quality, non-random pairs cannot establish a 2-point non-inferiority margin. H2 was not evaluated by the retained aggregate screening harness; H3 was not supported; routing and component attribution remain open. The \$429.96 estimate is fixed-mixture API-inference savings, not population-level TCO. The defensible conclusion is therefore strong but bounded: the complete B3 package reduces repeated long-context transmission in these controlled task sets, while quality, causal components, router utility, and real-repository impact require a powered preregistered study.

Data and code availability

The research code, configurations, and original experiment outputs are available at the immutable commit cited in Ref. [34]. The article repository contains the Flash-Lite replication inputs, SHA-256 manifest, portable evidence snapshot, and figure-generation script. The script recomputes E3 row-level aggregates, paired diagnostics, context summaries, E4 aggregate screening tables, and E5 extrapolations; E2/E6 summaries are imported with hashes rather than recomputed. Because complete generated answers are absent from the retained `runs.json`, pass/fail labels and original external-test executions cannot be independently replayed from the article artifact alone. The evidence snapshot records these boundaries and the analysis-script hash.

Declarations

Funding: no external funding was reported. **Competing interests:** the author declares no competing interests. **Ethics:** the study uses public programming benchmarks and does not involve human participants or personal data. **Author contributions:** D.P. designed the study, implemented the systems, executed the experiments, audited the evidence, and wrote the manuscript.

References

- [1] H. Sun and S. Zeng, “Introspection of Thought Helps AI Agents,” arXiv preprint arXiv:2507.08664, 2025.
- [2] A. J. Ko, R. DeLine, and G. Venolia, “Information Needs in Collocated Software Development Teams,” in *Proc. 29th Int. Conf. Software Engineering*, 2007, pp. 344–353.
- [3] C. Parnin and S. Rugaber, “Resumption Strategies for Interrupted Programming Tasks,” *Software Quality Journal*, vol. 19, no. 1, pp. 5–34, 2011.
- [4] M. K. Goncalves, C. R. B. de Souza, and V. M. Gonzalez, “Collaboration, Information Seeking and Communication,” *J. Universal Computer Science*, vol. 17, no. 11, pp. 1529–1550, 2011.
- [5] J. Wei *et al.*, “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” in *NeurIPS*, vol. 35, 2022, pp. 24824–24837.
- [6] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” *ICLR*, 2023.
- [7] A. Madaan *et al.*, “Self-Refine: Iterative Refinement with Self-Feedback,” *NeurIPS*, 2023.

- [8] N. Shinn *et al.*, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *NeurIPS*, 2023.
- [9] Z. Wang *et al.*, “Unleashing Cognitive Synergy in Large Language Models: A Task-Solving Agent through Multi-Persona Self-Collaboration,” in *Proc. NAACL-HLT*, 2024, pp. 257–279, doi:10.18653/v1/2024.naacl-long.15.
- [10] Y. Dong, X. Jiang, Z. Jin, and G. Li, “Self-Collaboration Code Generation via ChatGPT,” *ACM Trans. Software Engineering and Methodology*, vol. 33, no. 7, art. 189, pp. 1–38, 2024, doi:10.1145/3672459.
- [11] Z. Zeng *et al.*, “S2-MAD: Breaking the Token Barrier to Enhance Multi-Agent Debate Efficiency,” in *Proc. NAACL-HLT*, 2025, pp. 9393–9408, doi:10.18653/v1/2025.naacl-long.475.
- [12] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, “Agentless: Demystifying LLM-based Software Engineering Agents,” arXiv:2407.01489, 2024.
- [13] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, “Language Models Don’t Always Say What They Think: Unfaithful Explanations in Chain-of-Thought Prompting,” *NeurIPS*, 2023.
- [14] G. Weiss, *Multiagent Systems*. MIT Press, 1999.
- [15] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed. Wiley, 2009.
- [16] S. Hong *et al.*, “MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework,” *ICLR*, 2024.
- [17] C. Qian *et al.*, “ChatDev: Communicative Agents for Software Development,” *ACL*, 2024.
- [18] T. Guo *et al.*, “Large Language Model based Multi-Agents: A Survey of Progress and Challenges,” arXiv:2402.01680, 2024.
- [19] C. Qian *et al.*, “Scaling Large Language Model-based Multi-Agent Collaboration,” arXiv:2406.07155, 2024.
- [20] C. E. Jimenez *et al.*, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” *ICLR*, 2024.
- [21] J. Yang *et al.*, “SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering,” *NeurIPS*, 2024.
- [22] M. Chen *et al.*, “Evaluating Large Language Models Trained on Code,” arXiv:2107.03374, 2021.

- [23] J. Austin *et al.*, “Program Synthesis with Large Language Models,” arXiv:2108.07732, 2021.
- [24] H. Jiang *et al.*, “LongLLMLingua: Accelerating and Enhancing LLMs in Long Context Scenarios via Prompt Compression,” *ACL*, 2024.
- [25] Z. Pan *et al.*, “LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression,” *ACL Findings*, 2024.
- [26] H. Jia, E. T. Barr, and S. Mechtaev, “Compressing Code Context for LLM-based Issue Resolution,” arXiv:2603.28119, 2026.
- [27] Z. Bi *et al.*, “Iterative Refinement of Project-Level Code Context for Precise Code Generation with Compiler Feedback,” arXiv:2403.16792, 2024.
- [28] C.-Y. Su and C. McMillan, “Do Code LLMs Do Static Analysis?,” arXiv:2505.12118, 2025.
- [29] M. Sepidband, H. Taherkhani, S. Wang, and H. Hemmati, “Enhancing LLM-Based Code Generation with Complexity Metrics,” arXiv:2505.23953, 2025.
- [30] ISO/IEC 25010:2023, “Systems and software engineering — SQuaRE — Product quality model,” ISO, 2023.
- [31] T. J. McCabe, “A Complexity Measure,” *IEEE Trans. Software Engineering*, vol. SE-2, no. 4, pp. 308–320, 1976.
- [32] F. Wilcoxon, “Individual Comparisons by Ranking Methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
- [33] S. Holm, “A Simple Sequentially Rejective Multiple Test Procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.
- [34] D. Privezentsev, “INoT_Research: reproducible experiments for Hybrid-INoT,” GitHub repository, commit 010211ee5775185e8330ab6cde06e912c2adcb9e, 2026. [Online]. Available: https://github.com/daniil-novel/INoT_Research. Accessed: 28 July 2026.
- [35] OpenRouter, “Gemini 3.1 Pro Preview — pricing,” 2026. [Online]. Available: <https://openrouter.ai/google/gemini-3.1-pro-preview/pricing>. Accessed: 26 July 2026.
- [36] OpenRouter, “Gemini 3.1 Flash Lite Preview — pricing,” 2026. [Online]. Available: <https://openrouter.ai/google/gemini-3.1-flash-lite/pricing>. Accessed: 26 July 2026.